Project Explanation Guide

Project: Multi-Brand Smartphone Web Scraper
Domain: Data Engineering & Analytics
Internship: CodeAlpha — Task 1: Web Scraping
Target Source: Flipkart India (E-commerce)

---

1. PROJECT OVERVIEW & OBJECTIVE

This project automates the extraction, transformation, and structuring of consumer electronics listing data from public e-commerce web pages into a production-ready, clean CSV dataset. While modern e-commerce sites employ nested HTML hierarchies and non-uniform specification lists, this pipeline systematically scrapes, normalizes, and validates technical hardware parameters—such as brand, device model, cleaned price, RAM, ROM, and display attributes—across 13 market-leading mobile brands.

2. ARCHITECTURE & EXECUTION PIPELINE

```
[Target Brand Search URLs] ➔ [HTTP Requests (Custom Headers)] ➔ [DOM Parsing (BeautifulSoup)]
➔ [Regex Pattern Matching & Type Casting] ➔ [Deduplication & CSV Export (Pandas)]
```

Stage 1: Request Management & Anti-Throttling

- **Browser Headers:** Injects realistic browser headers (`User-Agent` , `Accept-Language` , `Accept`) to mimic human browsing and prevent bot detection/HTTP 403 errors.
- **Polite Crawling:** Enforces an automated delay of `time.sleep(1.2)` between sequential requests to adhere to crawling ethics and prevent server rate-limiting.

Stage 2: Dynamic DOM Parsing

- Queries Flipkart's search endpoint sequentially across 13 major brands: *Samsung, Apple, Vivo, Oppo, Realme, Xiaomi, OnePlus, Motorola, Poco, iQOO, Infinix, Google Pixel,* and *Nothing*.
- Employs the selector `soup.select("div[data-id]")` to target individual product container cards.
- Extracts the primary product title from the authoritative image `alt` attribute.

Stage 3: Specification Extraction via Regular Expressions (Regex)

- **Numeric Price Sanitization:** Detects raw price strings using `₹\s*[\d,]+` , removes all non-digit characters via `re.sub(r"[^\d]", "", raw_price)` , and casts directly to integer (`int`).
- **Hardware Identification:** Loops through nested list items (`ul li`) using targeted regular expressions:
  - **RAM:** Parsed via `r"(\d+\s*GB)\s*RAM"` .
  - **Storage (ROM):** Parsed via `r"(\d+\s*(?:GB|TB))\s*ROM"` .
  - **Display:** Filtered via keyword matching (`Display` , `Resolution` , `inch` , `FHD+` , `AMOLED`).
- **Fallback Protection:** Assigns `"N/A"` to missing properties (e.g., Apple devices that omit RAM) to prevent execution errors.

Stage 4: Data Hygiene & Export

- Applies `df.drop_duplicates(subset=["Model_Name"])` to eliminate duplicate listings.
- Saves the structured records to `Mobile_data.csv` using `utf-8-sig` encoding to maintain character integrity across Excel, Power BI, and Tableau.

3. DATASET SCHEMA & STRUCTURE

Field Name	Data Type	Description	Sample Value
Brand	String	Queried manufacturer brand name	Samsung
Model_Name	String	Full commercial title of the device	SAMSUNG Galaxy F15 5G (128 GB)
Price_INR	Integer	Clean numeric selling price in Indian Rupees	12999
RAM	String	Device active memory specification	6 GB
Storage	String	Internal storage capacity (ROM)	128 GB
Screen_Display	String	Display size and screen technology	16.51 cm (6.5 inch) FHD+ Display

4. ENGINEERING CHALLENGES & SOLUTIONS

- **Inconsistent Specification Layout:** Different manufacturers present specifications in varying list sequences. Instead of fragile index-based referencing (`specs[0]`), the pipeline evaluates all items against conditional regex patterns and keyword filters.
- **String Currency Artifacts:** E-commerce prices contain currency markers (`₹`) and commas (`,`). Sanitizing these during ingestion guarantees numerical types ready for statistical modeling without secondary processing.
- **Edge-Case Variations (e.g., Apple):** Apple listings do not advertise RAM. The script implements fallback capture logic checking for isolated ROM entries while setting missing attributes to `"N/A"` .

5. PRESENTATION & INTERVIEW TALKING POINTS

30-Second Elevator Pitch

> *"For Task 1 of my CodeAlpha Data Analytics Internship, I engineered an automated web scraping pipeline using Python, BeautifulSoup, and Pandas. The script queries 13 leading smartphone brands on Flipkart, parses nested DOM elements with regular expressions, extracts clean numeric pricing along with RAM, ROM, and display specs, and produces a deduplicated, analytics-ready CSV dataset."*

Walkthrough Script (Step-by-Step)

- **Ingestion Strategy:** *"I iterated over 13 brand targets, passing realistic HTTP headers and throttling delays (1.2s) to maintain stable, ethical scraping."*
- **DOM Navigation:** *"Using BeautifulSoup with CSS selectors (`div[data-id]`), the script locates individual product cards and extracts clean model titles directly from image attributes."*
- **Regex Transformation:** *"Because technical specifications are grouped in arbitrary bullet points, I applied regex pattern matching to extract RAM, storage, and screen metrics dynamically."*
- **Analytics Output:** *"The final dataset is deduplicated, type-cast, and saved in* `utf-8-sig` *format, making it directly consumable for exploratory data analysis (EDA)."*

6. DOWNSTREAM ANALYTICAL USE CASES

- **Price Tier Segmentation:** Classifying models into Budget (< ₹15,000), Mid-Range (₹15,000–₹35,000), and Flagship (> ₹35,000) segments.
- **Storage Cost Analysis:** Calculating the marginal cost per additional gigabyte of storage across brands.

- **Display Technology Penetration:** Measuring the distribution of AMOLED and FHD+ panels across various price tiers.